

An API for habits and routines.

A practical reference for the HTTP endpoints of authentication and the API Gateway, with usage examples, authentication, ownership-based authorization and service integration.

From client to microservices

HTTP/REST at the edge · Go and chi in the Gateway · gRPC with TLS between services · Node.js in the authentication service.

Documented domains

Authentication, users, social, habits, routines and statistics.

Main stack

Go · Node.js · gRPC · Protocol Buffers · PostgreSQL · Redis · Docker.

How the API is organized

The client uses HTTP. The Gateway authenticates calls and forwards domain operations to internal services over gRPC. The Auth service is a separate Node.js API that issues access and refresh tokens.

COMPONENT	TECHNOLOGY / ROLE	LOCAL PORT
Gateway	Go + chi · HTTP/REST	8084
Auth Service	Node.js + Express · tokens and authentication	3000 internal
User Service	gRPC over TLS	8080 internal
Social Service	gRPC over TLS	8081 internal
Habit / Routine Service	gRPC over TLS	8082 internal
Stats Service	gRPC over TLS	8083 internal
Nginx	Reverse proxy for Auth and Gateway	80

Direct Gateway

`http://localhost:8084`

Use it to call Gateway endpoints during local development.

Via Nginx

`http://localhost`

The proxy forwards Auth and Gateway routes through port 80.

Authentication flow

1. Create an account or log in to the Auth Service.
2. On protected Gateway routes, send the `Authorization` header with the Bearer scheme followed by the `accessToken`.
3. When the access token expires, send the `refreshToken` to renew the access token.

Runtime prerequisite: the Gateway validates the token using `JWT_SECRET`. Configure the Gateway container with the same secret used by the Auth Service; do not include the real secret value in documentation, examples or repositories.

General response format

```
{
  "timestamp": "2026-10-09T15:04:05Z",
  "api_version": "v1",
  "duration_ms": 12,
  "message": "habit created",
  "status": "success",
  "data": { },
  "http": { "method": "POST", "url": "/api/v1/habit/" }
}
```

On error, the response contains `status: "error"`, `message` and usually `error`. Some Auth errors may use a different format.

Auth Service

Auth routes go through Nginx at `http://localhost`. The full public paths keep the `/auth` prefix.

METHOD	ROUTE	BODY / NOTES
POST	<code>/api/v1/auth/signup</code>	<code>{"name": "Ana", "email": "ana@example.com", "password": "strong-password", "tel": "..."} </code> name, email and password required; tel optional.
POST	<code>/api/v1/auth/login</code>	<code>{"email": "ana@example.com", "password": "strong-password"}</code>
POST	<code>/api/v1/auth/refresh</code>	<code>{"refreshToken": "<refresh-token>"}</code>
POST	<code>/api/v1/auth/logout</code>	Authorization header with access token; revokes the user's refresh token.
GET	<code>/api/v1/auth/me</code>	Authorization header with access token; returns the authenticated user's ID.

Login example

```
curl -X POST http://localhost/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "ana@example.com", "password": "strong-password"}'
```

The login/signup response includes `accessToken` and `refreshToken`. The access token expires in 30 minutes; the refresh token in 7 days.

Internal port: Auth listens on port 3000, but the production Compose does not publish it directly on the host. Use the Nginx URL above.

Auxiliary Auth routes

METHOD	ROUTE	NOTES
GET	<code>/helpers/lifeCheck</code>	Health check (internal Auth route).
GET	<code>/users/me</code>	Authenticated user data; requires a token (internal Auth route).
GET	<code>/users/data/{id}</code>	Look up a user by ID (internal Auth route).

The auxiliary routes are mounted in the Auth Service, but are not forwarded by an Nginx location in the current configuration; port 3000 is also not published by the production Compose.

Auth uses JWT and Redis for refresh tokens. Never share real tokens in collections or public documentation.

Users and social

Gateway base: `http://localhost:8084`. All routes in this section require the `Authorization` header with the Bearer scheme and the access token issued by Auth.

Users · prefix `/api/v1/user`

METHOD	ROUTE	PARAMETER / BODY
GET	<code>/api/v1/user/GetUserByID/{id}</code>	User ID in the URL.
POST	<code>/api/v1/user/GetUsersByIDs</code>	<code>{"ids":["ID_1","ID_2"]}</code>
POST	<code>/api/v1/user/SearchUsers</code>	<code>{"name":"Ana","email":"ana@example.com"}</code> ; optional filters.
PUT	<code>/api/v1/user/EditUser/{id}</code>	<code>{"name":"Ana Souza","email":"new@example.com"}</code> ; optional fields. The <code>{id}</code> must be the authenticated user's.
PUT	<code>/api/v1/user/EditPassword/{id}</code>	<code>{"new_password":"new-password"}</code> . The <code>{id}</code> must be the authenticated user's.
DELETE	<code>/api/v1/user/DeleteUser/{id}</code>	User ID in the URL. You can only delete your own account.

Social · prefix `/api/v1/social`

METHOD	ROUTE	PARAMETER / BODY
POST	<code>/api/v1/social/StartFollowing</code>	<code>{"followee_id":"OTHER_USER_ID"}</code>
POST	<code>/api/v1/social/Unfollow</code>	<code>{"followee_id":"USER_ID"}</code>
GET	<code>/api/v1/social/ListFollowers/{id}</code>	Target user ID in the URL.
GET	<code>/api/v1/social/ListFollowing/{id}</code>	Target user ID in the URL.

The following user is taken from the token; do not send `follower_id`. The listing routes explicitly receive the ID of the user whose followers/following are being queried.

Ownership-based authorization: `EditUser`, `EditPassword` and `DeleteUser` only act on the authenticated user's own account; an `{id}` different from the token's is rejected. Details in section 07.

In the routes above, replace each placeholder in braces, such as `{id}`, with a real test ID.

Habits

Gateway base: `http://localhost:8084`. Habit and routine IDs are returned in the creation responses. Dates are sent in RFC 3339 format, for example `2026-10-09T15:00:00Z`.

Habits · prefix `/api/v1/habit` — the authenticated user's habits only

METHOD	ROUTE	PARAMETER / BODY
POST	<code>/api/v1/habit/</code>	<code>{"name": "Walk", "description": "30 min", "image_url": "https://..."};</code> name required.
GET	<code>/api/v1/habit/</code>	Lists the authenticated user's habits.
GET	<code>/api/v1/habit/{id}</code>	Habit ID. Only returns habits owned by the authenticated user.
GET	<code>/api/v1/habit/routine/{id}</code>	Routine ID. The routine and the returned habits must belong to the authenticated user.
PUT	<code>/api/v1/habit/{id}</code>	<code>{"name": "Updated walk"};</code> partial edit. Own habit only.
DELETE	<code>/api/v1/habit/{id}</code>	Habit ID. Own habit only.
POST	<code>/api/v1/habit/{id}/complete</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code>
DELETE	<code>/api/v1/habit/{id}/complete</code>	Same <code>completed_at</code> body to undo.
GET	<code>/api/v1/habit/{id}/logs</code>	<code>?start_date=2026-10-01T00:00:00Z&end_date=2026-10-10T00:00:00Z</code> ID and dates required (RFC 3339).

Shared reading · prefix `/api/v1/habit/shared`

METHOD	ROUTE	BEHAVIOR
GET	<code>/api/v1/habit/shared/{id}</code>	Fetches a habit by ID, even if it belongs to another person. NEW
GET	<code>/api/v1/habit/shared/routine/{id}</code>	Lists the habits associated with the routine. Returns only habits whose owner is the same as the routine's. NEW

Reading other users' data: `GET /habit/{id}` and `GET /habit/routine/{id}` return only the authenticated user's resources. To read another person's habit or routine, use the `/shared` routes, which are read-only and require authentication.

Completion dates and history filters use RFC 3339 format.

Routines

Gateway base: `http://localhost:8084`. Routines belong to the authenticated user. You can attach existing habits and record completions with a date.

Routines · prefix `/api/v1/routine` — the authenticated user's routines only

METHOD	ROUTE	PARAMETER / BODY
POST	<code>/api/v1/routine/</code>	<code>{"name": "Morning routine"}</code>
GET	<code>/api/v1/routine/</code>	Lists the authenticated user's routines.
GET	<code>/api/v1/routine/{id}</code>	Routine ID. Only returns routines owned by the authenticated user.
PUT	<code>/api/v1/routine/{id}</code>	<code>{"name": "New routine"}</code> . Own routine only.
DELETE	<code>/api/v1/routine/{id}</code>	Routine ID. Own routine only.
POST	<code>/api/v1/routine/{id}/habits</code>	<code>{"habit_id": "HABIT_ID"}</code> . The routine and the habit must belong to the authenticated user.
DELETE	<code>/api/v1/routine/{id}/habits/{habitID}</code>	Routine and habit IDs in the path. Both must belong to the authenticated user.
POST	<code>/api/v1/routine/{id}/complete</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code>
DELETE	<code>/api/v1/routine/{id}/complete</code>	Same <code>completed_at</code> body to undo.
GET	<code>/api/v1/routine/{id}/logs</code>	?start_date=2026-10-01T00:00:00Z&end_date=2026-10-10T00:00:00Z ID and dates required (RFC 3339).

Shared reading · prefix `/api/v1/routine/shared`

METHOD	ROUTE	BEHAVIOR
GET	<code>/api/v1/routine/shared/{id}</code>	Fetches a routine by ID, even if it belongs to another person. NEW

Reading other users' data: GET `/routine/{id}` returns only the authenticated user's routines. To fetch another person's routine, use `/shared/{id}` (read-only, requires authentication).

The undo-completion endpoints use DELETE with a JSON body. Make sure your HTTP client preserves the body on DELETE requests.

Habit and Routine are distinct domains in the API and share the internal gRPC service.

Statistics, errors and integration

Gateway base: `http://localhost:8084`. Statistics are tied to the authenticated user. There is no need to send the user ID in the URL or body.

Statistics · prefix `/api/v1/stats`

METHOD	ROUTE	PARAMETER / BODY
POST	<code>/api/v1/stats</code>	Creates the user's statistics; no body.
GET	<code>/api/v1/stats</code>	Fetches statistics; no body.
DELETE	<code>/api/v1/stats</code>	Removes statistics; no body.
POST	<code>/api/v1/stats/habits/{id}/completions</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code> . The habit must belong to the token's user.
DELETE	<code>/api/v1/stats/habits/{id}/completions</code>	Same <code>completed_at</code> body to undo. Habit owned by the token's user.
POST	<code>/api/v1/stats/routines/{id}/completions</code>	<code>{"completed_at": "2026-10-09T15:00:00Z"}</code> . The routine must belong to the token's user.
DELETE	<code>/api/v1/stats/routines/{id}/completions</code>	Same <code>completed_at</code> body to undo. Routine owned by the token's user.

The **stats-service** confirms that the habit or routine belongs to the token's user before recording or undoing the completion.

Common errors

STATUS	MEANING	SUGGESTED ACTION
400	Invalid JSON body, missing required field or invalid argument.	Check format, fields and dates.
401	Missing or invalid token in the Authorization header.	Log in and send the access token using the Bearer scheme.
404	Resource not found (depending on the service).	Check the ID and whether the resource exists.
413	Body above the route limit.	Reduce the JSON size.
500	Internal error, unavailable dependency or incomplete configuration.	Check service logs and configuration.

Testing best practices

1. Use a disposable development account and records.
2. Keep the IDs returned by signup, habit creation and routine creation.
3. Configure environment variables in Postman for the base URL and token; do not export real tokens.
4. To go through Nginx, replace the Gateway host with `http://localhost`; port 8084 is direct access to the Gateway.

Security: JWT tokens validated at the Gateway, refresh tokens in Redis, gRPC with TLS between services and ownership-based resource authorization (section 07). This guide describes the API as observed in the code.

Authorization and resource ownership

The authenticated user only reads and changes their own resources. The user ID always comes from the token, never from the client.

Summary by domain

DOMAIN	ROUTES	RULE APPLIED
User	PUT EditUser/{id}, PUT EditPassword/{id}, DELETE DeleteUser/{id}	The {id} in the URL must be the authenticated user's.
Habit	GET/PUT/DELETE /habit/{id}, POST/DELETE /habit/{id}/complete, GET /habit/{id}/logs, GET /habit/routine/{id}	Only the authenticated user's habits. In routine/{id}, the routine and the returned habits must belong to the user.
Routine	GET/PUT/DELETE /routine/{id}, POST/DELETE /routine/{id}/complete, GET /routine/{id}/logs	Only the authenticated user's routines.
Routine ↔ habit	POST /routine/{id}/habits, DELETE /routine/{id}/habits/{habitID}	The routine and the habit must belong to the authenticated user.
Statistics	POST/DELETE /stats/habits/{id}/completions, POST/DELETE /stats/routines/{id}/completions	The stats-service confirms the habit/routine belongs to the token's user.
Shared reading	GET /habit/shared/{id}, GET /habit/shared/routine/{id}, GET /routine/shared/{id}	Read-only; lets you view resources from other owners.

New routes · /shared

METHOD	ROUTE	BEHAVIOR
GET	/api/v1/routine/shared/{id}	Fetches a routine by ID, even if it belongs to another person. NEW
GET	/api/v1/habit/shared/{id}	Fetches a habit by ID, even if it belongs to another person. NEW
GET	/api/v1/habit/shared/routine/{id}	Lists the habits associated with the routine. Returns only habits whose owner is the same as the routine's. NEW

To read other users' habits and routines, use the /shared routes. The remaining habit and routine routes return only the authenticated user's resources.

How to test

1. Create two users, A and B, and a habit and a routine for user B.
2. With A's token, try PUT, DELETE and complete on B's habit and routine. All attempts are rejected.
3. With A's token, try EditUser, EditPassword and DeleteUser using B's ID. All are rejected.
4. With A's token, call GET /habit/shared/{id} on B's habit. It returns the habit.
5. With A's token, call GET /habit/{id} on B's habit. It is rejected.

Service contracts

The Gateway consumes these gRPC services over TLS. They are internal stack interfaces; they are not HTTP endpoints or public routes for the app.

SERVICE	ADDRESS INSIDE DOCKER	PROTO PACKAGE
UserService	user-service:8080	user
SocialService	social-service:8081	social
HabitService	habit-service:8082	habit
RoutineService	habit-service:8082	habit
StatsService	stats-service:8083	stats

Transport: gRPC over TLS, using project certificates. Ports 8080–8083 are internal to the Docker network; the production Compose does not publish them on the host. Port 8084 published by Compose serves the Gateway's HTTP, not gRPC.

Contract conventions

1. `string`, `bool`, integer types and lists are regular Protobuf fields.
2. `google.protobuf.Timestamp` represents an absolute UTC date/time; use RFC 3339 when converting to JSON.
3. Fields marked `optional` distinguish an absent value from an empty string.
4. The Gateway injects the authenticated user into domain calls when the REST method derives the ID from the token. The internal gRPC client receives the fields declared in the contract.

To test directly with a gRPC client outside Docker, you will need to publish/forward the appropriate port and configure trust in the CA certificate. This is not enabled by the default production Compose.

Contracts defined by the Protocol Buffers files in `backend/services/shared/proto`.

UserService

Package user · internal address user-service:8080 · TLS transport.

Model User

User { id: string, name: string, email: string }

RPC METHOD	REQUEST	RESPONSE
GetUserByID	GetUserByIDRequest { user_id: string }	GetUserByIDResponse { user: User }
GetUsersByIDs	GetUsersByIDsRequest { user_ids: string[] }	GetUsersByIDsResponse { users: User[] }
SearchUser	SearchUserRequest { name?: string, email?: string }	SearchUserResponse { user: User[] }
EditUser	EditUserRequest { user_id: string, name?: string, email?: string }	EditUserResponse { user: User }
EditPassword	EditPasswordRequest { user_id: string, new_password: string }	EditPasswordResponse { success: bool }
DeleteUser	DeleteUserRequest { user_id: string }	DeleteUserResponse { success: bool }

REST mapping: GetUserByID, GetUsersByIDs, SearchUser, EditUser, EditPassword and DeleteUser are exposed by the HTTP routes in the Users section. The change routes only accept the authenticated user's ID.

The REST batch lookup endpoint takes the ids field; the Gateway converts it to user_ids in the Protobuf request.

Contract: services/shared/proto/user/user.proto · internal port 8080

SocialService

Package `social` · internal address `social-service:8081` · TLS transport.

Model `User`

```
User { id: string, name: string, email: string }
```

RPC METHOD	REQUEST	RESPONSE
StartFollowing	StartFollowingRequest { follower_id: string, followee_id: string }	StartFollowingResponse { success: bool }
Unfollow	UnfollowRequest { follower_id: string, followee_id: string }	UnfollowResponse { success: bool }
ListFollowers	ListFollowersRequest { user_id: string }	ListFollowersResponse { users: User[] }
ListFollowing	ListFollowingRequest { user_id: string }	ListFollowingResponse { users: User[] }

REST mapping: the four methods are exposed by the Social routes in the Gateway section. For follow/unfollow, the Gateway fills `follower_id` with the authenticated user's ID and receives only `followee_id` from the client.

The service rejects invalid IDs, nonexistent users and attempts to follow oneself. Social gRPC errors are currently presented by the Gateway as HTTP 500.

Contract: `services/shared/proto/social/social.proto` · internal port 8081

HabitService

Package habit · internal address habit-service:8082 · TLS transport.

Models

Habit { id, user_id, name, description, image_url: string; created_at: Timestamp }

HabitLog { habit_id: string; completed_at: Timestamp }

RPC METHOD	REQUEST	RESPONSE
CreateHabit	{ habit: Habit }	{ habit: Habit }
GetHabitByID	{ habit_id: string }	{ habit: Habit }
ListHabitsByUser	{ user_id: string }	{ habits: Habit[] }
ListHabitsByRoutine	{ routine_id: string }	{ habits: Habit[] }
EditHabit	{ habit_id: string, name?: string, description?: string, image_url?: string }	{ habit: Habit }
DeleteHabit	{ habit_id: string }	{ success: bool }
MarkHabitCompleted	{ habit_id: string, completed_at: Timestamp }	{ success: bool }
UnmarkHabitCompleted	{ habit_id: string, completed_at: Timestamp }	{ success: bool }
GetHabitLogs	{ habit_id: string, start_date: Timestamp, end_date: Timestamp }	{ logs: HabitLog[] }

The Gateway sets user_id on creation and on per-user listing from the authenticated token. The other operations use the habit or routine ID given in the HTTP route.

Contract: services/shared/proto/habit/v4/habit.proto · internal port 8082

RoutineService

Package `habit` · shares the internal address `habit-service:8082` with `HabitService` · TLS.

Models

`Routine { id, user_id, name: string; created_at: Timestamp }`

`RoutineLog { routine_id: string; completed_at: Timestamp }`

RPC METHOD	REQUEST	RESPONSE
<code>CreateRoutine</code>	<code>{ routine: Routine }</code>	<code>{ routine: Routine }</code>
<code>GetRoutineByID</code>	<code>{ routine_id: string }</code>	<code>{ routine: Routine }</code>
<code>ListRoutinesByUser</code>	<code>{ user_id: string }</code>	<code>{ routines: Routine[] }</code>
<code>EditRoutine</code>	<code>{ routine_id: string, name?: string }</code>	<code>{ routine: Routine }</code>
<code>DeleteRoutine</code>	<code>{ routine_id: string }</code>	<code>{ success: bool }</code>
<code>AddHabitToRoutine</code>	<code>{ routine_id: string, habit_id: string }</code>	<code>{ success: bool }</code>
<code>RemoveHabitFromRoutine</code>	<code>{ routine_id: string, habit_id: string }</code>	<code>{ success: bool }</code>
<code>MarkRoutineCompleted</code>	<code>{ routine_id: string, completed_at: Timestamp }</code>	<code>{ success: bool }</code>
<code>UnmarkRoutineCompleted</code>	<code>{ routine_id: string, completed_at: Timestamp }</code>	<code>{ success: bool }</code>
<code>GetRoutineLogs</code>	<code>{ routine_id: string, start_date: Timestamp, end_date: Timestamp }</code>	<code>{ logs: RoutineLog[] }</code>

`user_id` for creation/listing comes from the identity authenticated at the Gateway. Link and completion operations receive the corresponding IDs and timestamps from the REST client.

Contract: `services/shared/proto/habit/v4/habit.proto` · internal port 8082

StatsService

Package stats · internal address stats-service:8083 · TLS transport.

Model UserStats

UserStats { user_id: string; completed_habits, completed_routines, current_habit_streak, longest_habit_streak, current_routine_streak, longest_routine_streak: int32; created_at, updated_at: Timestamp }

RPC METHOD	REQUEST	RESPONSE
CreateUserStats	{ user_id: string }	{ stats: UserStats }
GetUserStats	{ user_id: string }	{ stats: UserStats }
DeleteUserStats	{ user_id: string }	{ success: bool }
RegisterHabitCompletion	{ user_id: string, habit_id: string, completed_at: Timestamp }	{ success: bool }
UndoHabitCompletion	{ user_id: string, habit_id: string, completed_at: Timestamp }	{ success: bool }
RegisterRoutineCompletion	{ user_id: string, routine_id: string, completed_at: Timestamp }	{ success: bool }
UndoRoutineCompletion	{ user_id: string, routine_id: string, completed_at: Timestamp }	{ success: bool }

In Gateway calls, user_id comes from the token; the client provides the habit/routine ID and the completion date. The gRPC contract includes all three fields in the register and remove requests. The service confirms that the habit or routine belongs to the user_id before recording or undoing the completion.

Available calls

Creating, querying and removing the user's statistics; registering and reverting habit and routine completions. The corresponding HTTP endpoints are in the Statistics section.

The Stats service has versioned Proto definitions. This reference describes the current contract generated and consumed by the application; check the Proto file and generated code in case of divergence.

Contract: services/shared/proto/stats · internal port 8083